Starbucks: Improved Training for 2D Matryoshka Embeddings

Shengyao Zhuang*[3,1][0000-0002-6711-0955], Shuai Wang*[1][0000-0002-0726-5250], Fabio Zheng[1][0009-0008-0230-7969], Bevan Koopman[2,1][0000-0001-5577-3391], and Guido Zuccon[1,4][0000-0003-0271-5563]

[1] The University of Queensland, Brisbane, Australia
{s.zhuang, shuai.wang2, g.zuccon}@uq.edu.au,
fabio.zheng@student.uq.edu.au
[2] CSIRO, Brisbane, Australia
bevan.koopman@csiro.au
[3] Amazon, USA
[4] Google, Australia

Abstract. 2D Matryoshka training enables a single embedding model to produce sub-network representations across varying layers and embedding dimensions, offering flexibility under different computational and task constraints. However, its performance remains below that of individually trained models of comparable sizes. To address this, we propose **Starbucks**, a new training strategy for Matryoshka-style embedding models that combines structured fine-tuning with masked autoencoder (MAE) pre-training. During fine-tuning, we compute the loss over a fixed set of layer-dimension pairs, ordered from small to large, which significantly improves over random sub-network sampling and matches the performance of separately trained models. Our MAE-based pre-training further strengthens sub-network representations, providing a more robust backbone for downstream tasks. Experiments on both in-domain (semantic similarity and passage retrieval) and out-of-domain (BEIR) benchmarks show that Starbucks consistently outperforms 2D Matryoshka models and matches or exceeds the performance of individually trained models, while maintaining high efficiency and flexibility. Ablation studies validate our loss design, the benefits of SMAE pre-training, and demonstrate Starbucks' applicability across backbones. We further show that depth- and width-wise Starbucks variants encode complementary information, and that combining them yields further gains with minimal latency overhead via parallelization.[5]

Keywords: Information Retrieval · 2D Matryoshka Embeddings

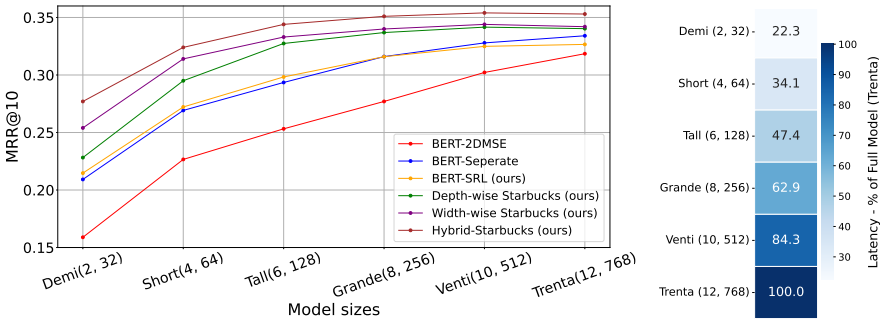


Fig. 1: Performance and efficiency of Starbucks models on MSMARCO. Left: MRR@10 across 2D Matryoshka configurations with Depth-width model sizes as SizeName (layers, embedding dim). Right: Latency (%) vs. Full DPR model, averaged over 10K queries and 8.8M passages using standard faiss index search.

1 Introduction

Matryoshka representation learning (MRL) [13] is a training technique that enables embedding models to support multiple dimensional granularities, allowing adaptability to different downstream resource constraints. Recently, the 2D Matryoshka Sentence Embeddings (2DMSE)[6] method was proposed to further enhance flexibility by enabling elastic embedding generation across both embedding dimensions and model layers [16]. This approach allows a single embedding model to replace multiple models trained with different size configurations, reducing training and deployment costs.

However, prior work has shown that the effectiveness of embeddings generated by 2DMSE sub-networks (using fewer layers or lower dimensions) is significantly lower to that of individually trained models targeting the same sizes [26]. Figure 1 confirms this degradation, highlighting a key limitation in existing 2DMSE training. In contrast, separately training multiple small-scale sub-models, despite higher training cost and inference memory use, remains preferred due to (1) significantly higher embedding quality, (2) the one-time nature of training, (3) often limited number of configurations required in practice.

To address this, we propose **Starbucks**, a new 2DMSE training framework comprising two components: **Starbucks Masked Autoencoding** (SMAE) for pre-training and **Starbucks Representation Learning** (SRL) for fine-tuning. Starbucks computes loss over a curated set of target layer-dimension pairs, covering a spectrum of subnetwork sizes. **SRL** promotes effective learning across these configurations, named after Starbucks cup sizes: Demi, Short, Tall, Grande, Venti, and Trenta (see Figure 2). Our results show that Starbucks matches the performance of separately trained models, making Matryoshka learning prac-

* Equal contribution.
[5] Code at https://github.com/ielab/Starbucks.
[6] Also known as Espresso Sentence Embeddings (ESE).

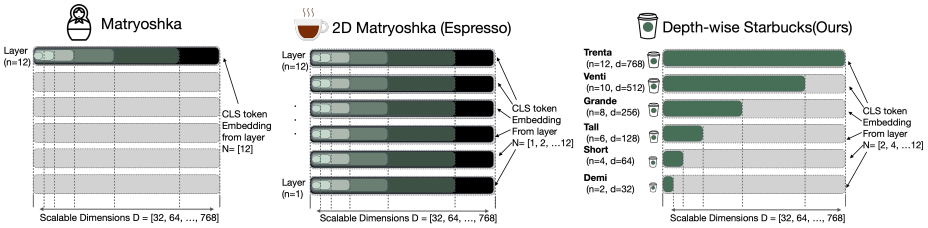


Fig. 2: Starbucks (right) provides a single model with flexible configurations in terms of embedding dimensionality and network depth. Also shown are the original Matryoshka model (left) and its extension, 2D Matryoshka (center). Here, we only use Depth-wise Starbucks to illustrate the approach.

tical. **SMAE** integrates ideas from MatFormer [8] and MAE-style embedding pre-training [9,29,25,32,17], applying masked autoencoding to representations from multiple layer-dimension pairs to prepare the model for SRL fine-tuning.

We evaluate Starbucks against 2DMSE and separately trained models on both in-domain (STS and passage retrieval) and out-of-domain (BEIR) tasks. Results show that: (1) Starbucks significantly improves over 2DMSE and even outperform the separately trained models while being more training time and inference memory efficient; (2) it generalizes well across architectures (BERT, E5); and (3) the training strategy supports both depth-wise and width-wise configurations. These variants produce complementary embeddings, especially in smaller subnetworks, and their hybridization yields further gains. As the configurations are independent, they can be applied in parallel, enabling scalable and efficient elastic embedding generation.

2 Related Work

Transformer-based text embedding models are highly effective across various NLP tasks, such as semantic text similarity [19,22,11,15] and passage retrieval [23,12,30,31]. The MRL approach aims to make embedding models more efficient and practical by utilising sub-dimensions of the embeddings to calculate the training loss [13]. After MRL training, all corresponding embeddings sub-dimensions can effectively represent the text, enabling more efficient inference for applications [14] with adaptive algorithms [21].

The original MRL method enabled adaptability only across output embedding dimensions. 2DMSE extends this by incorporating embeddings from multiple model layers, allowing adaptation to both embedding dimensions and model sizes (i.e., number of layers) [16]. Our proposed **Starbucks** similarly supports elastic embedding across dimensions and layers, but focuses on a fixed set of layer-dimension pairs from small to large. We show that this targeted training strategy is more effective than 2DMSE. Figure 2 illustrates the architectural differences between Starbucks and prior MRL approaches.

In addition to MRL for embedding models, other works have explored nested or elastic architectures in unsupervised language modeling. For instance, MatFormer introduced a nested transformer block for decoder-only language models [8]. Drawing inspiration from MatFormer, we propose **Width-wise Starbucks**, which reduces the dimensionality of internal layers instead of the number of layers, providing a complementary axis to depth-wise scaling. Separately, we introduce **Starbucks Masked Autoencoding (SMAE)**, a Matryoshka-style pre-training strategy tailored to embedding models, inspired by masked autoencoding approaches developed for dense retrieval [9,29,25,32,17].

3 Method

The Starbucks framework enables efficient and flexible representation learning by jointly training multiple sub-models within a single encoder. These sub-models vary in size and computational cost, and are designed to support diverse deployment-time requirements. Starbucks consists of two components: (1) Starbucks Representation Learning (SRL) for fine-tuning on downstream tasks, and (2) Starbucks Masked Autoencoding (SMAE) for pre-training the encoder.

At a high level, each sub-model in Starbucks is characterized by: (1) A **final embedding dimension** d, which determines the size of the vector used for retrieval and directly affects the *search time* [7]; and (2) An **encoder model size**, which determines the cost of encoding query online and encoding passages offline. Figure 1 illustrates how Starbucks-sized sub-models can significantly reduce online latency for both encoding and retrieval, compared to using the full model. We explore two orthogonal approaches for reducing encoding cost: (1) **Depth-wise Starbucks**, where the number of transformer layers n is reduced (e.g., using only the first 2, 4, or 6 layers). This was used in the same way when the original 2D Matryoshka models were introduced [16]; (2) **Width-wise Starbucks**, where each layer is thinned by reducing the intermediate feed-forward dimension and attention hidden size (e.g., halving from 3072 to 1536 or 512). Note that even though cutting model width was investigated in Matformer [8], it was never investigated in the context of 2D Matryoshka models.

In this work, we focus primarily on the *depth-wise* variant for analysis and benchmarking. The *width-wise* variant is included as an ablation (Section 7) to examine trade-offs under constant depth but reduced per-layer complexity.

3.1 Starbucks Representation Learning (SRL)

Given an input text x, we define a sub-model by selecting the first n encoder layers and the first d dimensions of the final representation.[8] The encoder outputs token embeddings:

$$V_n^d = \text{encoder}_{1:n}(x)_{1:d} \in \mathbb{R}^{|x| \times d},$$
$$v_n^d = \text{pool}(V_n^d) \in \mathbb{R}^{1 \times d},$$

[7] During similarity search, as smaller dimension will reduce the computation required

[8] Described using Depth-wise Starbucks as an example.

where pool$(\cdot)$ is a pooling function (e.g., mean or CLS), and v_n^d is the resulting embedding. We define a set $S = \{(n_1, d_1), \ldots, (N, D)\}$ representing sub-model configurations from small to full size. For Depth-wise Starbucks with BERT-base (N=12, D=768), we use $S = \{(2, 32), (4, 64), (6, 128), (8, 256), (10, 512), (12, 768)\}$. The primary loss objective is:

$$\mathcal{L}_S = \frac{1}{|S|} \sum_{(n,d)\in S} \text{loss}(v_n^d; A),$$

where loss$(\cdot)$ is a task-specific contrastive loss computed over a training batch A containing positive and in-batch negative pairs.

To promote consistency across sub-models, we add a KL divergence loss between each sub-model (n, d) and the full model (N, D):

$$\mathcal{L}_{\text{KL}} = \frac{1}{|S|} \sum_{(n,d)\in S} \text{KLD}(s_n^d, s_N^D),$$

where s_n^d denotes the probability distribution (similarity logits normalized via softmax) computed over the training batch A. In this formulation, the full-sized model s_N^D serves as the target (teacher) distribution. We specifically use this direction to ensure that the smaller sub-models (students) are penalized for deviating from the ranking manifold of the complete model. The total SRL loss is:

$$\mathcal{L}_{\text{SRL}} = \mathcal{L}_S + \mathcal{L}_{\text{KL}}.$$

Depth-wise vs. Width-wise Variants. The **depth-wise** variant reduces the number of transformer layers n while keeping widths fixed. The **width-wise** variant fixes $n = N$ and reduces intermediate and attention dimensions. Both use a shared embedding dimension d for downstream retrieval (see Section 7).

3.2 Starbucks Masked Autoencoding(SMAE)

To provide a stronger initialization for SRL, we propose **SMAE**, an adaptation of masked language modeling pretraining for multi-submodel setups. The architecture uses a shared encoder and a lightweight decoder.[9] Given input x, we generate masked variants:

$$x_{\text{enc}} = \text{Mask}(x, p_{\text{enc}}),$$
$$x_{\text{dec}} = \text{Mask}(x, p_{\text{dec}}),$$

with $p_{\text{dec}} > p_{\text{enc}}$ to encourage the decoder to rely on the encoder. The CLS token in x_{dec} is replaced with the encoder output from submodel (n, d):

$$V_n^d = \text{encoder}_{1:n}(x_{\text{enc}})_{1:d},$$
$$\boldsymbol{x}_{\text{dec}}^{\text{cls}} = \text{pool}(V_n^d \cdot W_{1:d}),$$

[9] Following RetroMAE, we adopt a one-layer decoder derived from BERT.

where $W \in \mathbb{R}^{D \times D}$ projects to full hidden size.

The masked language modeling losses for encoder and decoder are:

$$\mathcal{L}_{\text{enc}}(n, d) = -\frac{1}{|M_{\text{enc}}|} \sum_{i \in M_{\text{enc}}} \log P(x_i \mid V_n^d),$$

$$\mathcal{L}_{\text{dec}}(n, d) = -\frac{1}{|M_{\text{dec}}|} \sum_{i \in M_{\text{dec}}} \log P(x_i \mid \boldsymbol{x}_{\text{dec}}^{\text{cls}}, x_{\text{dec}}),$$

and the total SMAE loss over all submodels $(n, d) \in S$ is:

$$\mathcal{L}_{\text{SMAE}} = \frac{1}{|S|} \sum_{(n,d) \in S} \left[\mathcal{L}_{\text{enc}}(n, d) + \mathcal{L}_{\text{dec}}(n, d)\right].$$

After SMAE pre-training, the decoder is discarded. The encoder is then used as the initialization for Starbucks SRL. The SMAE strategy is applied to both depth-wise and width-wise settings, with S defined according to the corresponding sub-model architectural variant.

4 Experiment Setup

Starbucks Masked Autoencoding: We pretrain SMAE using the BERT-base architecture [7][10] on 10 billion tokens from the Fineweb dataset [20][11]. Training is conducted for one epoch with batch size 512, using AdamW (learning rate 1e-4, weight decay 0.05), cosine learning rate decay, and warmup ratio 0.05. We use the Hugging Face Transformers library to implement the training process [28]. The pre-trained SMAE checkpoint is available at https://huggingface.co/ielabgroup/bert-base-uncased-fineweb100bt-smae.

Fine-tuning and Evaluation: We organize our evaluation into two parts: **in-domain evaluation**, which assesses performance on tasks aligned with the training data, and **out-of-domain generalization**, which tests robustness across diverse unseen domains. **For In-domain Evaluation,** We evaluate Starbucks on two downstream tasks: **semantic text similarity** (STS) and **passage retrieval**. Fine-tuning is performed using our SRL tuning approach and the 2DMSE baseline, starting from either the original BERT or our SMAE-pretrained models. For **STS**, we follow standard practice by training on the MultiNLI [27] and SNLI [3] datasets, and evaluating on STS 2012–2016 [1], SICK-R [18], and STS-B [4]. Evaluation is based on Spearman's correlation. We use the Sentence Transformers library [22], extended to support SRL-style fine-tuning. For **passage retrieval**, we train on the MS MARCO Passage Ranking dataset [2] using Tevatron [10]. Evaluation is conducted on the MS MARCO development set using MRR@10, and on the TREC DL19 and DL20 benchmarks [6,5] using

[10] huggingface.co/google-bert/bert-base-uncased
[11] huggingface.co/datasets/HuggingFaceFW/fineweb

Table 1: In-domain evaluation on STS and Passage Retrieval, using averaged metrics across all Starbucks sizes: Spearman's correlation for STS, MRR@10 for MS MARCO (MS) dev, and NDCG@10 for DL19/DL20.

	Semantic Text Similarity Tasks								Retrieval Tasks		
Method	**STS-B**	**STS12**	**STS13**	**STS14**	**STS15**	**STS16**	**SICK-R**	**Average**	**MS.dev**	**DL19**	**DL20**
BERT	0.671	0.553	0.661	0.604	0.722	0.690	0.666	0.652	0.056	0.108	0.108
BERT-Separate	0.787	0.713	0.774	0.727	0.808	0.772	0.770	0.764	0.292	0.586	0.586
BERT-2DMSE	0.763	0.669	0.728	0.679	0.788	0.750	0.758	0.734	0.256	0.522	0.541
BERT-SRL	0.791	0.720	0.774	0.725	0.813	0.778	**0.777**	0.768	0.292	0.601	**0.613**
SMAE-SRL	**0.807**	**0.729**	**0.787**	**0.756**	**0.836**	**0.795**	0.776	**0.784**	**0.312**	**0.614**	0.604

nDCG@10. These benchmarks reflect both binary and graded relevance and are considered in-domain. The hyperparameters used for fine-tuning are summarized in https://github.com/ielab/Starbucks. **For Out-of-domain Generalization,** we evaluate models fine-tuned on MS MARCO across 13 public BEIR datasets [23] [12]. This setup tests Starbucks' ability to generalize to diverse domains in a zero-shot retrieval setting.

5 Results

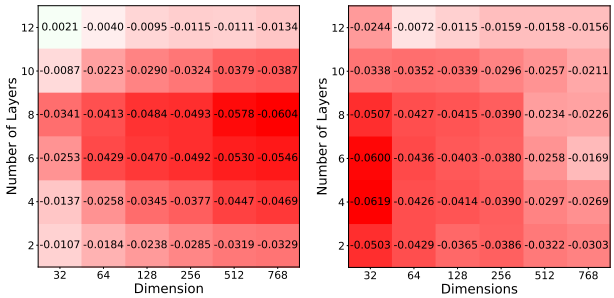
Results are presented in two parts: in-domain (Section 5.1), where test data match the training distribution, and out-of-domain (Section 5.2), which evaluates generalization to new retrieval domains.

5.1 In-domain Evaluation

Table 1 presents the average effectiveness across six Starbucks sizes, $S = \{(2, 32), (4, 64), \ldots, (12, 768)\}$, on STS and passage retrieval (per-size results are reported in Anonymous Github). We compare several training strategies: (i) full-size BERT models fine-tuned on each task and then sliced for evaluation (BERT), (ii) one BERT model fine-tuned separately for each Starbucks configuration (BERT-Separate), (iii) the 2D Matryoshka training method (BERT-2DMSE), (iv) our SRL fine-tuning method (BERT-SRL), and (v) the full Starbucks approach combining SMAE pre-training and SRL fine-tuning (SMAE-SRL).

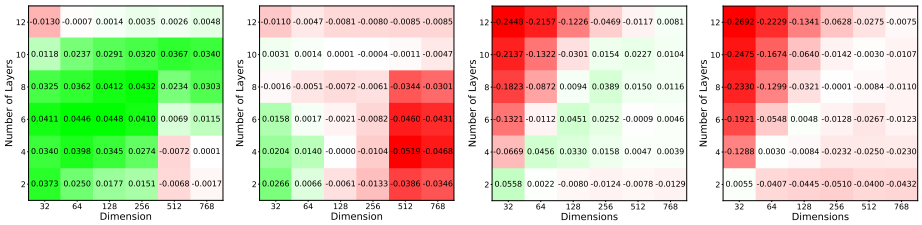
Overall, Starbucks outperforms all baselines on average, with the only exceptions occurring on SICK-R (STS) and DL20 (retrieval). SRL alone (BERT-SRL) already provides consistent improvements over 2DMSE and separate models across most datasets. These gains are further amplified when combined with SMAE pre-training.

[12] We exclude three private BEIR datasets, CQADupStack(due to evaluation issues), and MS MARCO dev (in-domain).



(a) STS Average (b) MARCO dev

Fig. 3: Comparison of 2DMSE and separately trained models on (a) STS (Spearman) and (b) MS MARCO dev (MRR@10). Red: separately trained models perform better.



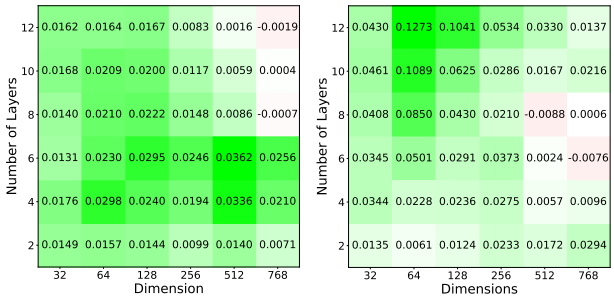
(a) STS Average (b) MARCO dev

Fig. 4: Comparison of SRL vs baselines on (a) STS (Spearman) and (b) MS MARCO dev (MRR@10). Left plots: SRL vs 2DMSE. Right plots: SRL vs separately trained models. Green: SRL better; Red: baseline better.

2DMSE vs. Separate Training Figure 3 shows the performance difference between 2DMSE and separately trained models. We find that separate training consistently yields better results across all configurations, with the exception of layer 12 and dimension 32. The gap is particularly pronounced for shallow layers, indicating that 2DMSE compromises effectiveness when distributing training across many subnetworks. Furthermore, we observe that layer depth has a greater impact on effectiveness than embedding dimension. A possible explanation lies in the original 2DMSE loss setup, where only one early layer and the final layer are used—giving the last layer disproportionate weight. This may weaken supervision for earlier layers[13].

SRL vs. Baselines Figure 4 compares SRL with both 2DMSE and separately trained models. SRL fine-tuning is performed in a single training run from the original BERT checkpoint, targeting six predefined Starbucks sizes. Despite this

[13] We also tried adding all layers in the loss, which slightly improved early layers but significantly hurt final-layer performance.



(a) STS (b) MS MARCO

Fig. 5: Comparison of SMAE-SRL and BERT-SRL on (a) STS (Spearman) and (b) MS MARCO dev (MRR@10). Green: SMAE-SRL performs better.

constraint, SRL generalizes well to nearby configurations and consistently outperforms 2DMSE on the STS task. We hypothesize that this is due to the complexity of the 2DMSE training objective, which may lead to suboptimal optimization and degraded performance across all configurations.

The most substantial gains are observed along the Starbucks diagonal—that is, when the number of layers and embedding dimensions match those seen during training—highlighting the benefit of explicitly training for these size pairs. When comparing SRL to upper-bound separately trained models, SRL achieves comparable, and in some cases higher effectiveness, particularly in smaller configurations with shallow depths. Performance on off-diagonal configurations (e.g., 12 layers with 32 dimensions) tends to be weaker, likely because SRL's training objective emphasizes the diagonal pairs, resulting in less effective generalization to mismatched sizes.

Impact of SMAE Pre-training Figure 5 presents a comparison between models fine-tuned with SRL using different initialization strategies: BERT versus SMAE. The results show that SMAE pre-training consistently outperforms the BERT baseline, particularly for smaller configurations with fewer layers and lower-dimensional embeddings. The only exception is in full-size models, where performance is marginally lower, potentially explaining why certain datasets, such as SICK-R, show slightly reduced effectiveness under SMAE. These findings suggest that SMAE, designed to learn compressed representations, is particularly well-suited for initializing sub-networks for SRL fine-tuning.

5.2 Out-of-domain Evaluation

To evaluate the generalization ability of Starbucks, we assess Starbucks variants and baseline models in a zero-shot setting: all models are fine-tuned on MS

Table 2: Out-of-domain evaluation on BEIR using average nDCG@10 across all Starbucks sizes.

	General		Scientific			Debate		Finance	Biomedical		Wiki		
Method	**Db.Ent.**	**Quora**	**Scid.**	**Scif.**	**Cli.F**	**Argu.**	**Touch.**	**Fiqa**	**T.Cvd**	**Nfcorp.**	**Fever**	**Hpqa**	**Avg.**
BERT	0.051	0.236	0.020	0.092	0.028	0.055	0.039	0.038	0.114	0.061	0.110	0.076	0.077
BERT-Separate	0.255	**0.782**	0.089	0.409	0.116	0.227	0.211	0.176	0.536	0.225	**0.566**	0.363	0.330
BERT-2DMSE	0.208	0.585	0.069	0.330	0.106	0.199	0.205	0.139	0.476	0.187	0.459	0.261	0.269
BERT-SRL	0.248	0.699	0.089	0.408	0.122	0.222	0.225	0.170	0.546	0.220	0.549	0.354	0.321
SMAE-SRL	**0.261**	0.656	**0.101**	**0.413**	**0.133**	**0.245**	**0.233**	**0.210**	**0.619**	**0.247**	0.562	**0.371**	**0.338**

Table 3: Impact of KL divergence loss $\mathcal{L}_{\text{KL}}$. STS-B Spearman's Correlation is used for STS task, and MARCO dev MRR@10 is used for Retrieval.

		Demi (2, 32)	Short (4, 64)	Tall (6, 128)	Grande (8, 256)	Venti (10. 512)	Trenta (12, 768)	Average
STS	w. $\mathcal{L}_{\text{KL}}$	0.7455	**0.7933**	**0.8202**	**0.8278**	**0.8292**	**0.8274**	**0.8073**
	w/o. $\mathcal{L}_{\text{KL}}$	**0.7463**	0.7925	0.8190	0.8274	0.8283	0.8269	0.8067
Retrieval	w. $\mathcal{L}_{\text{KL}}$	**0.2282**	**0.2950**	**0.3274**	**0.3369**	**0.3416**	0.3403	**0.3116**
	w/o. $\mathcal{L}_{\text{KL}}$	0.2205	0.2885	0.3162	0.3295	0.3372	**0.3405**	0.3054

MARCO and directly evaluated—without further adaptation—on 13 datasets from the BEIR benchmark. Average results are shown in Table 2 [14].

The results show that SRL matches the effectiveness of BERT models trained separately for each size. Notably, SRL requires only a single training run and supports multiple configurations at inference by adjusting layer-dimension parameters. With SMAE pre-training, SRL outperforms separately trained BERT variants on 11 of 13 datasets, demonstrating strong improvements in zero-shot generalization. These results confirm that the gains observed in in-domain settings transfer robustly to diverse retrieval tasks in BEIR.

6 Ablation: Impact of Starbucks Modeling Choices

In this section, we investigate variations in modeling choices and their impact on embedding quality through three key ablation studies. First, we examine the role of **KL divergence** during fine-tuning. Second, we evaluate the effect of different **pre-training strategies**. Finally, we test SRL fine-tuning applicability to a recent **state-of-the-art embedding backbone**, E5.

6.1 Impact of KL Divergence

We examine the effect of the KL divergence component in the SRL loss, defined as $\mathcal{L} = \mathcal{L}_S + \mathcal{L}_{\text{KL}}$, which encourages alignment between the full-size model embeddings and those of smaller layer-dimension configurations.

Results in Table 3 show that incorporating $\mathcal{L}_{\text{KL}}$ generally benefits performance across most Starbucks sizes. Notable exceptions include the Demi size on

[14] Full per-size comparison shown in https://github.com/ielab/Starbucks

Table 4: Impact of pre-training components. STS-B Spearman's Correlation is used for the STS task, and MARCO dev MRR@10 is used for the retrieval task.

		Demi (2, 32)	Short (4, 64)	Tall (6, 128)	Grande (8, 256)	Venti (10. 512)	Trenta (12, 768)	Average
STS	SMAE	0.7455	0.7933	0.8202	**0.8278**	**0.8292**	0.8274	**0.8073**
	- MAE	**0.7475**	**0.7968**	**0.8211**	0.8257	0.8288	0.8239	**0.8073**
	- Starbucks	0.7399	0.7675	0.7902	0.8134	0.8238	**0.8360**	0.7951
Retrieval	SMAE	0.2281	0.2949	**0.3273**	**0.3369**	**0.3416**	**0.3403**	**0.3116**
	- MAE	**0.2349**	**0.2972**	0.3237	0.3282	0.3339	0.335	0.3088
	- Starbucks	0.2255	0.2859	0.3098	0.3277	0.3375	**0.3403**	0.3044

Table 5: Impact of pre-training backbone. STS-B Spearman's Correlation is used for the STS task, and MARCO dev MRR@10 is used for the retrieval task.

		Demi (2, 32)	Short (4, 64)	Tall (6, 128)	Grande (8, 256)	Venti (10, 512)	Trenta (12, 768)	Average
STS	E5-base-v2	0.5893	0.5616	0.5778	0.5842	0.6910	0.8548	0.6431
	BERT-SMAE-SRL	**0.7455**	0.7933	**0.8202**	0.8278	0.8292	0.8274	0.8073
	E5-2DMSE	0.7050	0.7341	0.7607	0.7871	0.8072	**0.8673**	0.7769
	E5-SRL	0.7313	0.7728	0.7984	0.8243	**0.8452**	0.8627	0.8058
	E5-SMAE-SRL	0.7437	**0.7922**	0.8175	**0.8302**	0.8337	0.8331	**0.8084**
Retrieval	E5-base-v2	0.0015	0.0021	0.0042	0.0088	0.0097	**0.3606**	0.0645
	BERT-SMAE-SRL	0.2282	0.2950	**0.3274**	**0.3369**	0.3416	0.3403	**0.3116**
	E5-2DMSE	0.1330	0.2062	0.2429	0.2629	0.2905	0.2953	0.2385
	E5-SRL	0.2159	0.2793	0.3115	0.3336	**0.3425**	0.3503	0.3055
	E5-SMAE-SRL	**0.2309**	**0.3013**	0.3217	0.3333	0.3364	0.3369	0.3101

STS and the Trenta size on retrieval. Overall, the impact of $\mathcal{L}_{\text{KL}}$ is more substantial in the retrieval setting, while STS shows only marginal improvements.

6.2 Impact of Pre-training Approaches

To understand the effect of various pre-training strategies, we ablate components of the Starbucks pre-training method (Section 3.2) while keeping SRL fine-tuning fixed. We compare three pre-training configurations: (1) full Starbucks with Masked Autoencoding (MAE), (2) Starbucks without MAE, and (3) the original BERT model trained only with MLM.

As shown in Table 4, full Starbucks pre-training improves effectiveness across most model sizes, except for Trenta, which already aligns with the full-size training objective. Smaller models (e.g., Demi, Short) sometimes perform better without the MAE component, while larger ones (e.g., Grande, Venti, Trenta) benefit from its inclusion. These effects are more pronounced in retrieval than STS, likely because retrieval requires finer-grained representations to match queries against large corpora, whereas STS is a simpler sentence-pair task.

6.3 Applicability to SOTA Backbone

To evaluate the generality of our SRL fine-tuning approach, we apply it to E5 [24], a recent state-of-the-art (SOTA) embedding model. Table 5 presents the results of applying SRL and SMAE to the E5 backbone. We observe that

SRL remains effective even when used with more recent and highly optimized models. In some configurations, particularly smaller model variants, SRL even improves overall effectiveness.

However, we note a slight decrease in performance for the full model. We hypothesize that this drop is due to the full E5 model already being extensively tuned across a variety of datasets; applying SRL may shift the optimization landscape in a way that prioritizes consistency across sub-networks at the cost of peak full-model retrieval performance.

Notably, combining SRL with SMAE pre-training encourages stronger representation learning in earlier layers. This sometimes comes at the expense of later-layer expressiveness, but it reinforces that even standalone SRL is sufficient when applied to strong modern backbones. These findings suggest that Starbucks strategies can be effectively applied to SOTA architectures like E5, without requiring significant changes to their pre-existing optimization pipelines.

7 Depth-wise vs. Width-wise Starbucks

In this section, we investigate **Width-wise Starbucks**, a variant that reduces the dimensionality of the intermediate and attention layers within each transformer block, while preserving the full layer depth. This approach draws inspiration from MatFormer [8], and contrasts with **Depth-wise Starbucks**, examined in the previous sections, which reduces the number of transformer layers.

To ensure comparability between Depth-wise and Width-wise models, we scale the intermediate and attention layer dimensions in Width-wise Starbucks using a fixed division factor of six, matching the layer-step granularity used in Depth-wise configurations. Due to architectural constraints in BERT (e.g., the requirement for 12 attention heads), we further adjust the attention dimensions to ensure compatibility. Despite these adjustments, the parameter difference between corresponding Depth-wise and Width-wise models remains negligible—all under 0.2 million parameters ($<0.05\%$) [15].

Table 6 presents a direct comparison between Depth-wise and Width-wise Starbucks variants. Before SMAE pre-training, both variants show comparable average effectiveness. However, after applying SMAE, Width-wise Starbucks consistently outperforms Depth-wise Starbucks, particularly for smaller model sizes. We hypothesize that this improvement arises from the different types of contextualized representations induced by depth versus width reduction. Since these configurations alter the computation graph in distinct ways, their resulting embeddings are not functionally equivalent—especially in smaller sub-networks (e.g., Demi and Short).

Motivated by this observation, we explore a **hybrid approach** that fuses embeddings from paired Depth-wise and Width-wise models of the same size. As shown in Table 6, this approach provides the largest relative improvements for the smallest models, confirming the complementary nature of the two variants. As model size increases, the relative benefit diminishes, suggesting that

[15] Full configuration details presented in https://github.com/ielab/Starbucks

Table 6: Comparison of Depth- vs. Width-wise Starbucks (SRL and SMAE-SRL) on MS MARCO dev (MRR@10). *Hybrid* gains over the stronger of *Depth* or *Width* shown in parentheses. * indicates significant difference ($p < 0.05$).

Model	**SRL**			**SMAE-SRL**		
	Depth	Width	Hybrid	Depth	Width	Hybrid
Demi (2, 32)	0.215	0.209	0.253 (17.8%)	0.228	0.254*	0.277 (9.0%)
Short (4, 64)	0.272	0.276*	0.299 (8.4%)	0.295	0.314*	0.324 (3.2%)
Tall (6, 128)	0.298	0.298	0.319 (6.9%)	0.327	0.333	0.344 (3.2%)
Grande (8, 256)	0.316	0.311	0.330 (4.5%)	0.337	0.340	0.351 (3.1%)
Venti (10, 512)	0.325	0.323	0.338 (4.0%)	0.342	0.344	0.354 (3.1%)
Trenta (12, 768)	0.327	0.327	0.339 (3.6%)	0.340	0.342	0.353 (3.5%)
Avg.	0.292	0.291	**0.313** (7.1%)	0.312	0.321	**0.334** (4.0%)

larger sub-networks already capture sufficiently rich representations. Overall, this hybrid approach consistently improves performance across all model sizes. Importantly, because Depth-wise and Width-wise Starbucks configurations operate independently, hybridization can be performed in parallel with minimal additional inference cost, making the approach practical for real-world deployment where both effectiveness and efficiency are critical.

8 Conclusion

This paper highlights a key limitation of 2D Matryoshka Embedding models: despite their support for elastic embedding generation across layers and dimensions, their effectiveness remains well below that of separately trained models.

To address this, we propose **Starbucks**, a training framework that combines structured fine-tuning with masked autoencoding pre-training. Experiments on in-domain and out-of-domain tasks show that Starbucks outperforms 2D Matryoshka and matches, or even exceeds, the effectiveness of individually trained models, while remaining efficient and adaptable. Starbucks also yields a robust BERT-based backbone via SMAE for initializing downstream sub-models with or without SRL. Ablation studies confirm the value of SMAE and demonstrate generalization to modern architectures like E5. Additionally, depth-wise and width-wise variants offer complementary representations, and a simple hybrid strategy further boosts performance with minimal latency overhead.

Overall, Starbucks bridges the effectiveness gap in elastic embedding models compared to separately trained small models, while retaining adaptability. It also opens promising directions for future work, such as automatically selecting optimal layer-dimension configurations to balance performance and efficiency in practical applications like embedding-as-a-service systems.

Competing Interests The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Agirre, E., Banea, C., Cer, D., Diab, M., Gonzalez-Agirre, A., Mihalcea, R., Rigau, G., Wiebe, J.: SemEval-2016 task 1: Semantic textual similarity, monolingual and cross-lingual evaluation. In: Bethard, S., Carpuat, M., Cer, D., Jurgens, D., Nakov, P., Zesch, T. (eds.) Proceedings of the 10th International Workshop on Semantic Evaluation (SemEval-2016). pp. 497–511. Association for Computational Linguistics, San Diego, California (Jun 2016). https://doi.org/10.18653/v1/S16-1081, https://aclanthology.org/S16-1081
2. Bajaj, P., Campos, D., Craswell, N., Deng, L., Gao, J., Liu, X., Majumder, R., McNamara, A., Mitra, B., Nguyen, T., Rosenberg, M., Song, X., Stoica, A., Tiwary, S., Wang, T.: MS MARCO: A human generated machine reading comprehension dataset (2018), https://arxiv.org/abs/1611.09268
3. Bowman, S.R., Angeli, G., Potts, C., Manning, C.D.: A large annotated corpus for learning natural language inference. In: Màrquez, L., Callison-Burch, C., Su, J. (eds.) Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing. pp. 632–642. Association for Computational Linguistics, Lisbon, Portugal (Sep 2015). https://doi.org/10.18653/v1/D15-1075, https://aclanthology.org/D15-1075
4. Cer, D., Diab, M., Agirre, E., Lopez-Gazpio, I., Specia, L.: SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In: Bethard, S., Carpuat, M., Apidianaki, M., Mohammad, S.M., Cer, D., Jurgens, D. (eds.) Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017). pp. 1–14. Association for Computational Linguistics, Vancouver, Canada (Aug 2017). https://doi.org/10.18653/v1/S17-2001, https://aclanthology.org/S17-2001
5. Craswell, N., Mitra, B., Yilmaz, E., Campos, D.: Overview of the TREC 2020 deep learning track (2021), https://arxiv.org/abs/2102.07662
6. Craswell, N., Mitra, B., Yilmaz, E., Campos, D., Voorhees, E.M.: Overview of the TREC 2019 deep learning track (2020), https://arxiv.org/abs/2003.07820
7. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., Doran, C., Solorio, T. (eds.) Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). pp. 4171–4186. Association for Computational Linguistics, Minneapolis, Minnesota (Jun 2019). https://doi.org/10.18653/v1/N19-1423, https://aclanthology.org/N19-1423
8. Devvrit, Kudugunta, S., Kusupati, A., Dettmers, T., Chen, K., Dhillon, I., Tsvetkov, Y., Hajishirzi, H., Kakade, S., Farhadi, A., Jain, P.: MatFormer: Nested transformer for elastic inference (2023), https://arxiv.org/abs/2310.07707
9. Gao, L., Callan, J.: Condenser: a pre-training architecture for dense retrieval. In: Moens, M.F., Huang, X., Specia, L., Yih, S.W.t. (eds.) Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 981–993. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic (Nov 2021). https://doi.org/10.18653/v1/2021.emnlp-main.75, https://aclanthology.org/2021.emnlp-main.75
10. Gao, L., Ma, X., Lin, J., Callan, J.: Tevatron: An efficient and flexible toolkit for neural retrieval. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 3120–3124. SIGIR '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3539618.3591805, https://doi.org/10.1145/3539618.3591805

11. Gao, T., Yao, X., Chen, D.: SimCSE: Simple contrastive learning of sentence embeddings. In: Moens, M.F., Huang, X., Specia, L., Yih, S.W.t. (eds.) Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 6894–6910. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic (Nov 2021). https://doi.org/10.18653/v1/2021.emnlp-main.552, https://aclanthology.org/2021.emnlp-main.552
12. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: Webber, B., Cohn, T., He, Y., Liu, Y. (eds.) Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 6769–6781. Association for Computational Linguistics, Online (Nov 2020). https://doi.org/10.18653/v1/2020.emnlp-main.550, https://aclanthology.org/2020.emnlp-main.550
13. Kusupati, A., Bhatt, G., Rege, A., Wallingford, M., Sinha, A., Ramanujan, V., Howard-Snyder, W., Chen, K., Kakade, S., Jain, P., Farhadi, A.: Matryoshka representation learning. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) Advances in Neural Information Processing Systems. vol. 35, pp. 30233–30249. Curran Associates, Inc. (2022), https://proceedings.neurips.cc/paper_files/paper/2022/file/c32319f4868da7613d78af9993100e42-Paper-Conference.pdf
14. Lee, J., Dai, Z., Ren, X., Chen, B., Cer, D., Cole, J.R., Hui, K., Boratko, M., Kapadia, R., Ding, W., Luan, Y., Duddu, S.M.K., Abrego, G.H., Shi, W., Gupta, N., Kusupati, A., Jain, P., Jonnalagadda, S.R., Chang, M.W., Naim, I.: Gecko: Versatile text embeddings distilled from large language models (2024), https://arxiv.org/abs/2403.20327
15. Li, X., Li, J.: AoE: Angle-optimized embeddings for semantic textual similarity. In: Ku, L.W., Martins, A., Srikumar, V. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 1825–1839. Association for Computational Linguistics, Bangkok, Thailand (Aug 2024). https://doi.org/10.18653/v1/2024.acl-long.101, https://aclanthology.org/2024.acl-long.101
16. Li, X., Li, Z., Li, J., Xie, H., Li, Q.: ESE: Espresso sentence embeddings (2024), https://arxiv.org/abs/2402.14776
17. Lu, S., He, D., Xiong, C., Ke, G., Malik, W., Dou, Z., Bennett, P., Liu, T.Y., Overwijk, A.: Less is more: Pretrain a strong siamese encoder for dense text retrieval using a weak decoder. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 2780–2791 (2021)
18. Marelli, M., Menini, S., Baroni, M., Bentivogli, L., Bernardi, R., Zamparelli, R.: A SICK cure for the evaluation of compositional distributional semantic models. In: Calzolari, N., Choukri, K., Declerck, T., Loftsson, H., Maegaard, B., Mariani, J., Moreno, A., Odijk, J., Piperidis, S. (eds.) Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14). pp. 216–223. European Language Resources Association (ELRA), Reykjavik, Iceland (May 2014), http://www.lrec-conf.org/proceedings/lrec2014/pdf/363_Paper.pdf
19. Muennighoff, N., Tazi, N., Magne, L., Reimers, N.: MTEB: Massive text embedding benchmark. In: Vlachos, A., Augenstein, I. (eds.) Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics. pp. 2014–2037. Association for Computational Linguistics, Dubrovnik, Croatia (May 2023). https://doi.org/10.18653/v1/2023.eacl-main.148, https://aclanthology.org/2023.eacl-main.148

20. Penedo, G., Kydlíček, H., allal, L.B., Lozhkov, A., Mitchell, M., Raffel, C., Werra, L.V., Wolf, T.: The fineweb datasets: Decanting the web for the finest text data at scale (2024), https://arxiv.org/abs/2406.17557
21. Rege, A., Kusupati, A., S, S.R., Fan, A., Cao, Q., Kakade, S., Jain, P., Farhadi, A.: AdANNS: A framework for adaptive semantic search. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems. vol. 36, pp. 76311–76335. Curran Associates, Inc. (2023), https://proceedings.neurips.cc/paper_files/paper/2023/file/f062da1973ac9ac61fc6d44dd7fa309f-Paper-Conference.pdf
22. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). pp. 3982–3992. Association for Computational Linguistics, Hong Kong, China (Nov 2019). https://doi.org/10.18653/v1/D19-1410, https://aclanthology.org/D19-1410
23. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., Gurevych, I.: BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: Vanschoren, J., Yeung, S. (eds.) Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. vol. 1 (2021), https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/65b9eea6e1cc6bb9f0cd2a47751a186f-Paper-round2.pdf
24. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., Wei, F.: Text embeddings by weakly-supervised contrastive pre-training. arXiv preprint arXiv:2212.03533 (2022)
25. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., Wei, F.: SimLM: Pre-training with representation bottleneck for dense passage retrieval. In: Rogers, A., Boyd-Graber, J., Okazaki, N. (eds.) Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 2244–2258. Association for Computational Linguistics, Toronto, Canada (Jul 2023). https://doi.org/10.18653/v1/2023.acl-long.125, https://aclanthology.org/2023.acl-long.125
26. Wang, S., Zhuang, S., Koopman, B., Zuccon, G.: 2d matryoshka training for information retrieval. arXiv preprint arXiv:2411.17299 (2024)
27. Williams, A., Nangia, N., Bowman, S.: A broad-coverage challenge corpus for sentence understanding through inference. In: Walker, M., Ji, H., Stent, A. (eds.) Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). pp. 1112–1122. Association for Computational Linguistics, New Orleans, Louisiana (Jun 2018). https://doi.org/10.18653/v1/N18-1101, https://aclanthology.org/N18-1101
28. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., Rush, A.: Transformers: State-of-the-art natural language processing. In: Liu, Q., Schlangen, D. (eds.) Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 38–45. Association for Computational Linguistics, Online (Oct 2020). https://doi.org/10.18653/v1/2020.emnlp-demos.6, https://aclanthology.org/2020.emnlp-demos.6

29. Xiao, S., Liu, Z., Shao, Y., Cao, Z.: RetroMAE: Pre-training retrieval-oriented language models via masked auto-encoder. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing. pp. 538–548. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (Dec 2022). https://doi.org/10.18653/v1/2022.emnlp-main.35, https://aclanthology.org/2022.emnlp-main.35
30. Zhao, W.X., Liu, J., Ren, R., Wen, J.R.: Dense text retrieval based on pretrained language models: A survey. ACM Trans. Inf. Syst. **42**(4) (Feb 2024). https://doi.org/10.1145/3637870, https://doi.org/10.1145/3637870
31. Zhuang, S., Ma, X., Koopman, B., Lin, J., Zuccon, G.: PromptReps: Prompting large language models to generate dense and sparse representations for zero-shot document retrieval (2024), https://arxiv.org/abs/2404.18424
32. Zhuang, S., Shou, L., Pei, J., Gong, M., Ren, H., Zuccon, G., Jiang, D.: Typos-aware bottlenecked pre-training for robust dense retrieval. In: Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region. pp. 212–222. SIGIR-AP '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3624918.3625324, https://doi.org/10.1145/3624918.3625324